

83.Remove Duplicates from Sorted Linked List

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: June 19, 2026

Problem

Given the head of a **sorted** linked list, delete all duplicate values such that each element appears only **once**. Return the linked list (still sorted).

Key Insight

Since the list is **sorted**, duplicates are always **adjacent**. So we never need to look ahead more than one node — just compare each node with the previous one and skip if equal.

Approach

Use two pointers:

curr

prev

- curr traverses the list.
- prev tracks the last unique node.

When a duplicate is found:

prev.next = curr.next;

which removes the duplicate node.

Algorithm

1. Set:

prev = head;

curr = head.next;

2. Traverse while curr != null.

3. If values are equal:

prev.next = curr.next;

4. Else:

prev = curr;

5. Move:

curr = curr.next;

6. Return head.

Time Complexity & Space Complexity

TC : $O(n)$, Each node is visited once.

SC : $O(1)$, No extra space used.

The screenshot displays a LeetCode problem page for "83. Remove Duplicates from Sorted List". The problem description states: "Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well." Two examples are provided: Example 1 shows a linked list [1, 1, 2] being transformed into [1, 2]; Example 2 shows [1, 1, 2, 3, 3] being transformed into [1, 2, 3]. The solution is implemented in Java, showing the ListNode class and the deleteDuplicates method. The method uses two pointers, prev and curr, to traverse the list and skip duplicate nodes. The runtime is 0 ms, and the memory usage is 45.50 MB. The test result shows the solution is accepted for both test cases.

83. Remove Duplicates from Sorted List Solved

Easy Topics Companies

Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Example 1:

Input: head = [1,1,2]
Output: [1,2]

Example 2:

Input: head = [1,1,2,3,3]
Output: [1,2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].

Accepted 168 / 168 testcases passed
Sahil Khan submitted at Jun 19, 2026 22:50

Runtime 0 ms | Beats 100.00%
Memory 45.50 MB | Beats 50.19%

Code

```
1 /**  
2  * Definition for singly-linked list.  
3  * public class ListNode {  
4  *     int val;  
5  *     ListNode next;  
6  *     ListNode() {}  
7  *     ListNode(int val) { this.val = val; }  
8  *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }  
9  * }
```

```
10 class Solution {  
11     public ListNode deleteDuplicates(ListNode head) {  
12         if (head == null) return head;  
13         ListNode curr = head;  
14         ListNode prev = curr;  
15  
16         while (curr != null) {  
17             if (curr.val == prev.val) {  
18                 prev.next = curr.next;  
19             } else {  
20                 prev = curr;  
21             }  
22             curr = curr.next;  
23         }  
24         return head;  
25     }  
26 }
```

Testcase **Test Result**

Accepted Runtime: 0 ms

Case 1 **Case 2**

Input: head = [1,1,2,3,3]
Output: [1,2,3]